

ClaimTrek

A GPS territory-capture running game · Mobile Systems (MobSys) 2026 · Dr.-Ing. David Sommer

Participants

Name	Matrikelnummer
Sahil Lakhani	320806
Rhitik Vishnu Nimbalkar	320993

What the app does

ClaimTrek turns a run into a territory game. You go for a run, and when your GPS track forms a **closed loop**, you claim the ground inside it. The claim is measured in real square metres, so a bigger loop is worth more. Running a loop over another player's ground **takes the overlap from them**: their plot shrinks, or disappears entirely if you leave them almost nothing. The map is a live picture of who currently owns what, and the leaderboard ranks players by total area held.

A phone sitting in a car traces a perfect loop, so the app has to be hard to cheat. While you run it cross-checks your movement against the **accelerometer**, looking for the frequency signature of a running gait. A run that fails the check is still saved and still drawn on the map, but it is marked *unverified* and left out of the leaderboard — the app never silently discards someone's effort. The **barometer** records altitude throughout the run and produces the elevation profile shown on the summary and history screens.

A second mode, **Trek**, queries the Overpass (OpenStreetMap) REST API for real named hiking routes within 5 km and draws the chosen one on the map.

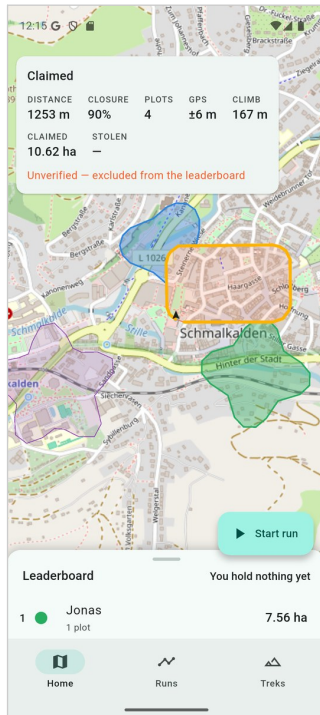
Players sign in with **Google** through Firebase Authentication, after which saved runs and the player's standing are published to **Cloud Firestore** under the real account, and the leaderboard folds remote players in with local ones. Signing in is optional: the local SQLite database stays the source of truth, so the game is fully playable with no account and no signal, and a failed upload costs a leaderboard update rather than a run.

How the exercise requirements are met

Requirement	In ClaimTrek
At least 3 Activities	Six screens: Home (map), Tracking, Run summary, Run history, Run detail, Treks + Trek detail.
Data to the next Activity and back	Tracking hands the closed loop and its claim preview to Run summary; the Save / Discard decision is returned and only then is anything written to storage.
CustomView	The elevation profile chart is drawn with a custom painter — axis labels, profile line and gradient fill are rendered directly onto the canvas.
RecyclerView / list adapter	Recycling lists with custom row layouts for run history, the leaderboard sheet and the trek list.
Two sensors	Barometer (altitude, climb, elevation profile) and accelerometer (gait anti-cheat). Also GPS, pedometer and compass.
Data storage	SQLite for territories, runs and the trail cache; key-value preferences for identity; Cloud Firestore for accounts, published runs and the leaderboard.
Networking / REST API	Overpass API for hiking routes (cached by map cell, as it is rate-limited); OpenStreetMap tiles; Firebase Authentication and Cloud Firestore.
Varied UI components	Bottom navigation bar, draggable bottom sheet, progress indicators, text field, snackbars, pop-up menu, cards and floating action button.

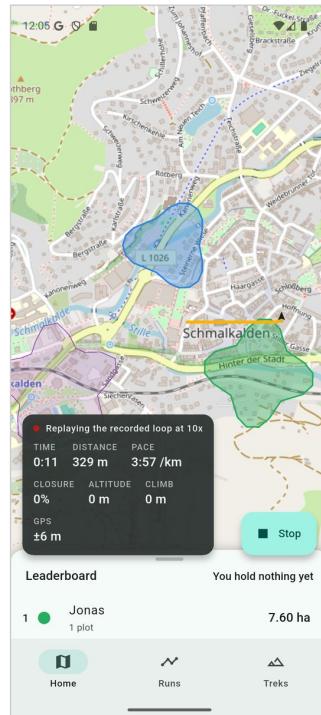
How the app looks

Screenshots taken on an Android emulator (Pixel 9a, API 36).



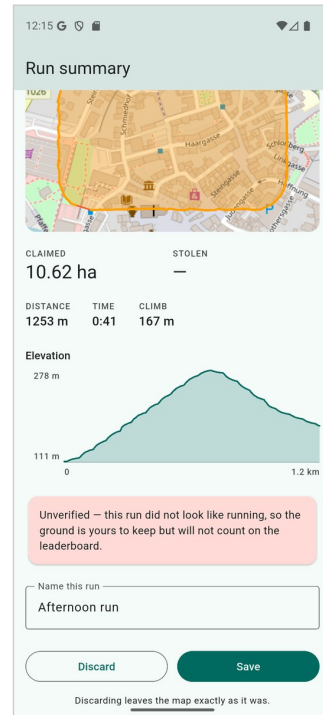
Home — the map

Claimed ground in orange, three rival plots around it. The counter shows the run just finished; the leaderboard sheet drags up from the bottom.



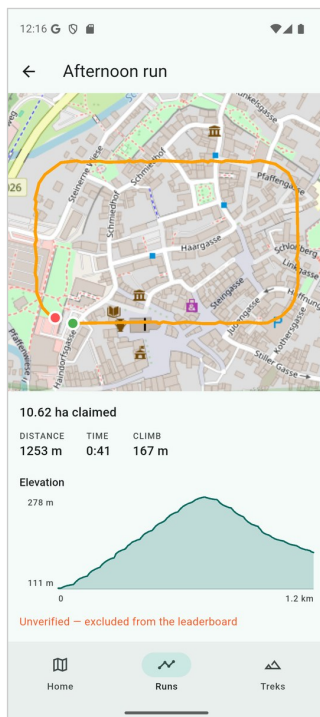
Tracking a run

The live counter: time, distance, pace, loop-closure progress, altitude, climb and GPS accuracy. The track is drawn as you move.



Run summary — CustomView

Area claimed and taken from rivals, and the elevation profile drawn by the custom painter. Nothing is written until Save.



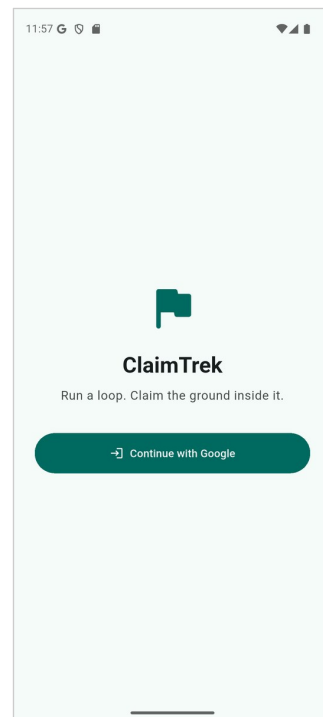
Run detail

A stored run reloaded from the database: the path redrawn with start and end markers, and the elevation profile rebuilt from the saved samples.



Treks — REST API

Named hiking routes near the runner, fetched live from the Overpass (OpenStreetMap) API and cached locally to respect its rate limit.



Sign in

Google sign-in through Firebase Authentication. The app is fully playable without an account, so this screen is never forced on the user.

Notes on the implementation

Framework used

The exercise specifies Android Studio with Kotlin. This project is written in **Flutter / Dart** and builds a native Android application (package `de.hsm.claimtrek`, minSdk 23, target API 36), which runs on the emulator and on a physical device. We are raising this openly rather than leaving it to be discovered, and are happy to discuss it.

Problems encountered

The geometry library did not work. Territory capture needs polygon overlay operations — intersection, union, difference. The library we chose first exposes all of these methods, but every one of them is an unimplemented stub that throws at run time. It failed in the most misleading way possible: because the error was caught and treated as "this territory is unaffected", stealing silently did nothing at all while appearing to succeed. We replaced it with a real clipping library and stopped wrapping geometry calls in catch-all handlers.

Degrees are not metres. Doing the geometry in latitude and longitude gives areas that are wrong, and two runs compared against each other shear apart. Every run is now projected into a local planar metre frame about a reference point, which is stored per territory so the exact frame can be rebuilt later. Area then reads directly in square metres.

A missing sensor must hide a feature, never break it. A sensor that is absent does not raise an error — its stream simply never delivers a value, which is indistinguishable from a sensor that is present but idle. Each sensor is therefore probed at start-up and any feature behind a missing one is hidden. This mattered for the anti-cheat in particular: a flat, silent accelerometer reads as "movement with no gait", i.e. a vehicle, so without the probe every run on a phone without the sensor would have been wrongly marked as cheating.

A plausible-looking GPS fix can still be wrong. Filtering each fix on its own accuracy and reported speed is all that can be done without history, and it misses the failure that actually distorts a claimed area: a fix eighty metres from the truth that reports six metre accuracy and a walking pace, which is what signal bouncing off buildings produces. Only the distance from the previous fix, measured against the time between them, reveals it. A second filter now does that, and gives up after repeated rejections so that a runner who genuinely did relocate is not locked out of their own track by a filter measuring from a position they had left.

Note on the screenshots

The Android emulator's virtual barometer always reports exactly 1013.25 hPa, the standard sea-level constant, which the barometric formula maps to precisely 0 m. The elevation chart was therefore perfectly flat on the emulator. For the screenshots above the virtual sensor was driven through a realistic pressure curve so the chart shows genuine relief. On real hardware the value comes from the device's own barometer, or from GPS altitude where no barometer exists.

Testing

The project carries **225 automated tests** covering the geometry engine, loop detection, the anti-cheat analysis, GPS filtering, cloud synchronisation, storage and the user interface, plus an architecture test that fails the build if framework code is imported into the pure calculation modules. The full capture pipeline — GPS fixes, loop closure, claim, steal, save — is driven end to end from a recorded GPX track, so it can be tested indoors without running a real loop.